МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 1 ЗА ТЕМОЮ:

Реалізація rule-based системи

Група ІАВ - 21
Курс 2
Студентка Кодовбецька В. І.
Дата оформлення 14.10.2026

Київ - 2026

Лабораторна робота №1. Реалізація rule-based системи

Завдання

1. Реалізувати rule-based систему в Python.

2. Завантажити базу фактів і правил.

3. Реалізувати механізм логічного виведення.

Очікуваний результат: працюючий прототип rule-based системи, який може робити висновки на основі фактів і правил.

Хід виконання завдання:

У цій роботі реалізовано ядро експертної системи керування мікрокліматом розумного будинку, спроєктованої в практичних роботах №1–3. Програму написано на Python без сторонніх бібліотек, тому вона запускається на будь-якому комп'ютері з Python 3.10 і новіше. Факти й правила не вшиті в код, а лежать окремими JSON-файлами, щоб у наступних лабораторних їх можна було доповнювати, не переписуючи програму.

1. Реалізація rule-based системи в Python

Проєкт розбито на окремі компоненти, як у блок-схемі з практичної роботи №1: база знань (папка data), робоча пам'ять і механізм виведення (папка engine) та точка запуску main.py.

```
smart-home-ics/
├─ data/
│  ├─ rules.json          24 правила (ПР3 + картка)
│  ├─ facts.json          база фактів з ПР2
│  ├─ initial_state.json  початковий стан пристроїв
│  └─ scenarios.json      тестові ситуації
├─ engine/
│  ├─ working_memory.py   робоча пам'ять
│  ├─ rules.py            правила та перевірка умов
│  └─ inference.py        механізм виведення
└─ main.py                запуск із консолі
```

1) Робоча пам'ять (working_memory.py) – зберігає поточні факти у вигляді «об'єкт.атрибут → значення», наприклад «Кімната.температура → 28».
2) Правила (rules.py) – клас Rule зберігає назву, пріоритет, вагу, умову й дії правила, вміє перевірити умову, виконати дії та пояснити, яких фактів бракує, щоб правило спрацювало.
3) Механізм виведення (inference.py) – функція run, яка циклічно застосовує правила до робочої пам'яті.
4) Точка запуску (main.py) – завантажує файли, прогоняє сценарії й виводить хід міркувань і стан пристроїв.

2. Завантаження бази фактів і правил

Базу фактів записано у форматі «об'єкт – атрибут – значення», як у практичній роботі №2. Кожен факт – окремий рядок JSON:

```
{"object": "Кімната", "attribute": "температура", "value": 28}
{"object": "Картка", "attribute": "вставлена", "value": "ні"}
{"object": "Живлення", "attribute": "джерело", "value": "мережа"}
```

Перед фактами завантажується початковий стан (initial_state.json): режим «комфортний», криза «ні», усі пристрої вимкнені. Тестові ситуації в scenarios.json беруть за основу facts.json і змінюють у ньому лише потрібні факти.

Правила перенесено з формального запису в додатку Б практичної роботи №3, де кожне правило має кілька передумов і один висновок. Умова – це або окреме порівняння, або група умов «all» (І) чи «any» (АБО). Для прикладу правило R18:

```
{"id": "R18", "name": "Високий CO2", "salience": 80, "weight": 0.8,
 "if": {"all": [
   {"fact": "Кімната.рівень_CO2", "op": ">",  "value": 1000},
   {"fact": "Мешканці.є_люди",    "op": "==", "value": "так"},
   {"fact": "Режим.криза",        "op": "==", "value": "ні"},
   {"fact": "Живлення.джерело",   "op": "==", "value": "мережа"}]},
 "then": [{"set": "Вікна.стан", "value": "відкрито"}]}
```

Якщо в робочій пам'яті немає факту, на який посилається умова, умова вважається невиконаною, і програма не падає.

Окремо доопрацьовано визначення присутності людей. Раніше воно трималося лише на датчику руху, який не відрізнить людину від кота чи штори, що ворухнулась від вітру. Тому за порадою викладача додано картку-ключ, яку мешканець вставляє, коли він удома. Тепер присутність визначається карткою (правила R1 і R2), а датчик руху працює на охорону: якщо картки немає, а рух є, система розбирається, чи це тварина (R3), можливий грабіжник (R4) чи, при відкритому вікні, просто вітер або штора (R5). Ці п'ять правил мають найвищий пріоритет 110, бо від факту «є люди» залежать інші правила. Разом із 19 правилами з практичної роботи №3 база налічує 24 правила.

3. Механізм логічного виведення

Реалізовано пряме виведення (forward chaining): система йде від фактів до висновків. Кожен цикл складається з трьох кроків, показаних на рисунку 1.

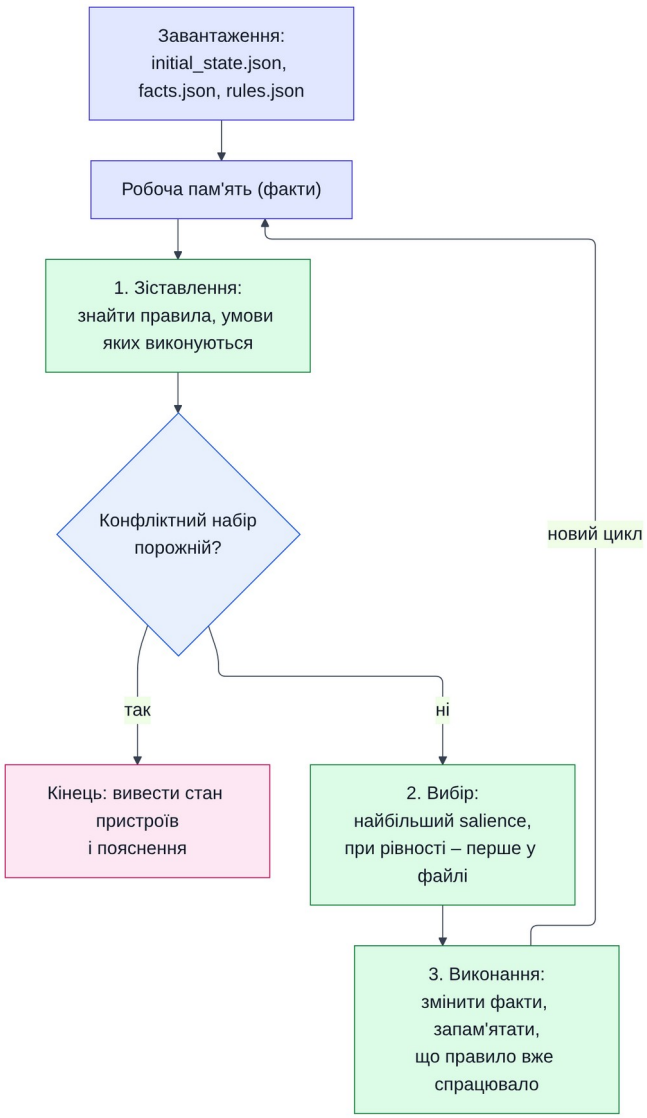


Рисунок 1 – Цикл роботи механізму виведення

1) Зіставлення. Перевіряються умови всіх правил, і ті, що виконуються, потрапляють у конфліктний набір.
2) Вибір. З набору береться правило з найбільшим salience. Якщо пріоритети однакові, спрацьовує те, що стоїть раніше у файлі.

3) Виконання. Права частина правила змінює факти в робочій пам'яті, і починається новий цикл, бо нові факти можуть увімкнути або вимкнути інші правила.

Кожне правило спрацьовує лише один раз (рефракція), тому система не може зациклитись. Виведення зупиняється, коли конфліктний набір порожній. Для кожного циклу програма друкує конфліктний набір, обране правило, факти, через які воно спрацювало, і зміни, тобто пояснює своє рішення. З ключем --explain вона ще й показує, яких умов не вистачило правилам, що не спрацювали.

Результати роботи

Систему перевірено на семи сценаріях з файлу scenarios.json.

Сценарій	Спрацювали правила	Результат
1. Приклад з ПР2: літо, 28 °C, картки немає, вдома тварини	R2, R3, R19, R20, R24	рух від тварини, охолодження, поїлка, економний режим
2. Блекаут посеред спекотного дня	R1, R10–R15	усі пристрої вимкнено, режим сну
3. Обстріл, мешканець удома, CO_2 1300 ppm	R1, R6–R9	рециркуляція, ізоляція вікон, криза, ізольований режим; R18 і R21 заблоковано
4. Мережа повернулась, зима, 16 °C, сніг	R1, R16, R22, R23	режим комфортний, обігрівач і тепла підлога увімкнені
5. Ніч, мешканець поворухнувся, картка вставлена	R1	мешканець удома, тривоги немає
6. Картки немає, тварин немає, вікна закриті, є рух	R2, R4, R24	сигналізація: тривога
7. Те саме, але вікно відкрите	R2, R5, R24	рух, ймовірно, від вітру або штори, тривоги немає

Перший сценарій – це база фактів із практичної роботи №2. Картки немає, тому людей удома немає, а рух дає кіт. Далі за пріоритетом спрацювали правила для тварин у спеку і наприкінці економний режим (рисунок 2).

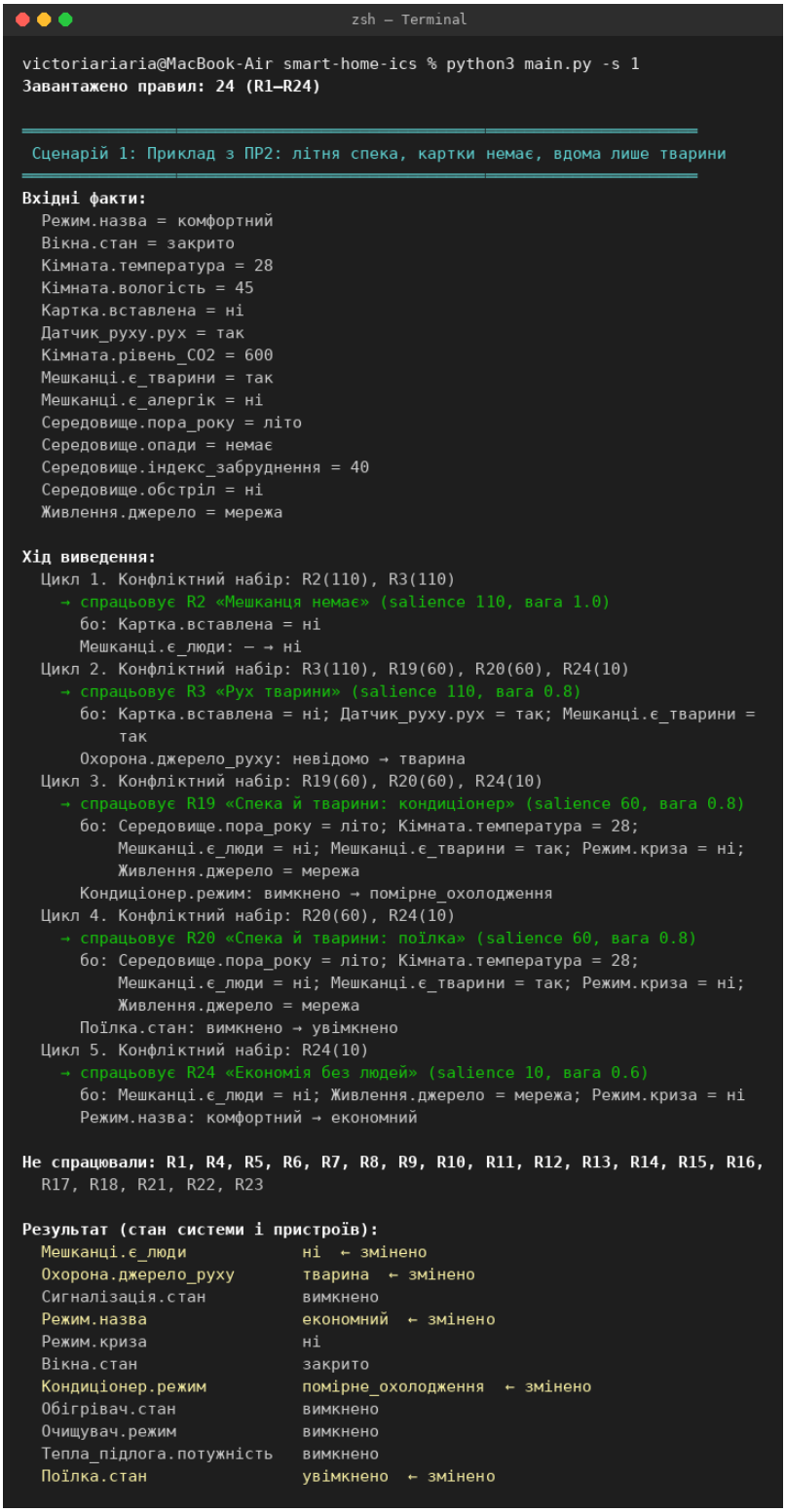


Рисунок 2 – Сценарій 1: приклад з практичної роботи №2

У третьому сценарії правила безпеки спрацювали одне за одним: R6 увімкнуло рециркуляцію, R7 ізолювало вікна, R8 записало факт «криза = так», а R9 уже на його основі перевело систему в ізольований режим. Це

ланцюжок виведення. Через кризу R18 (відкрити вікна через CO_2) і R21 (охолодження) не спрацювали, і ключ --explain показує саме цю причину (рисунок 3).

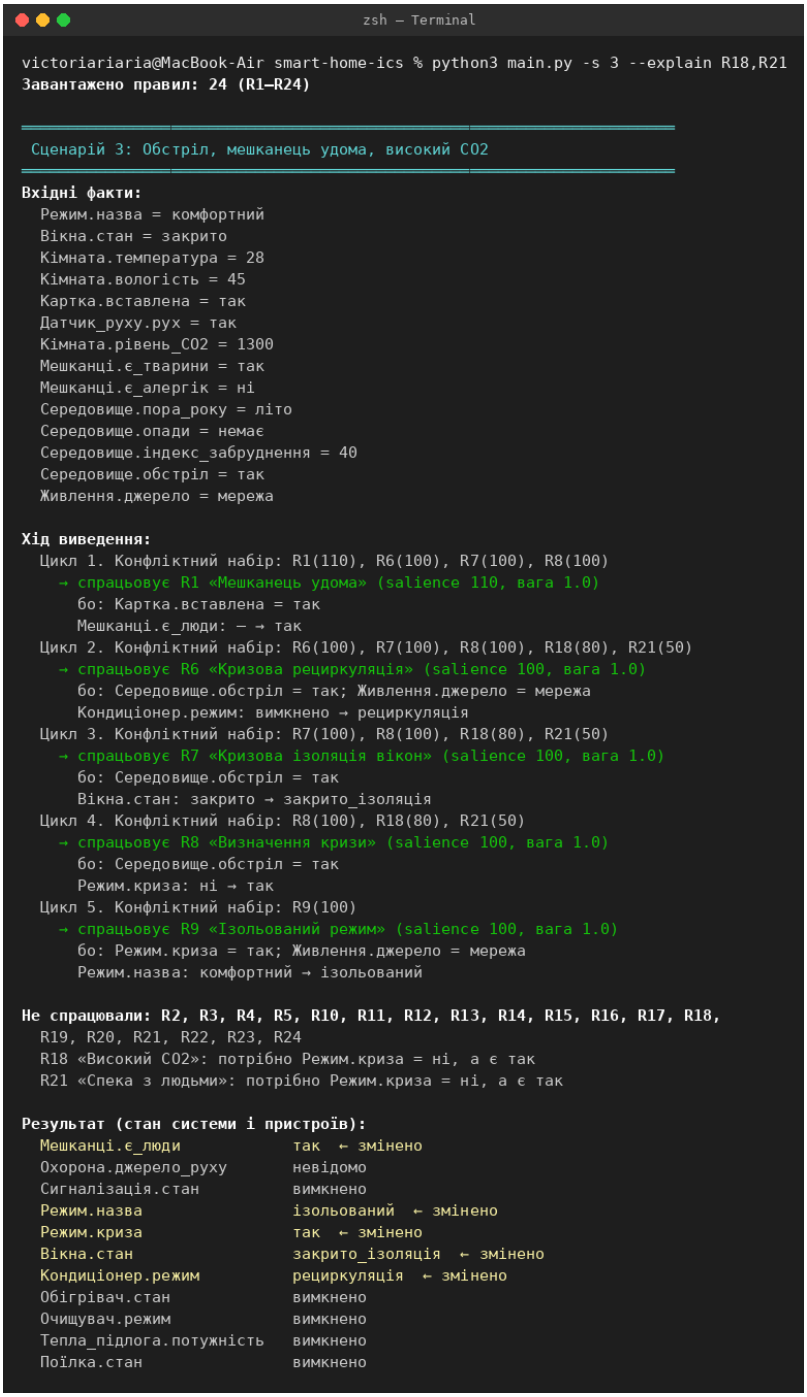


Рисунок 3 – Сценарій 3: обстріл, запуск з поясненням

Четвертий сценарій показує кілька циклів поспіль: після визначення присутності R16 повернуло комфортний режим після сну, потім спрацювали R22 (холодно, мешканець удома) і R23 (сніг) (рисунок 4).

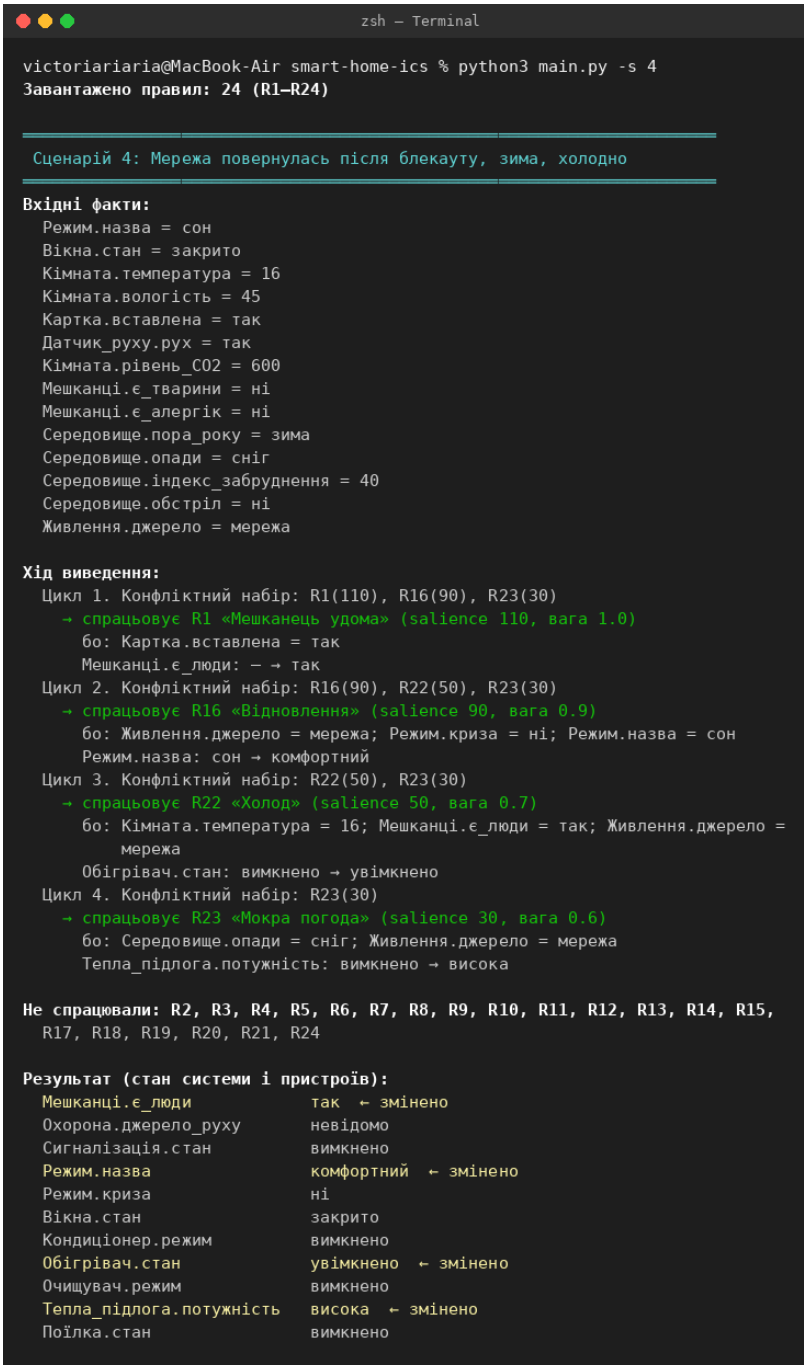


Рисунок 4 – Сценарій 4: відновлення мережі взимку

Сценарії 5–7 перевіряють картку-ключ. Вночі мешканець поворухнувся, але картка вставлена, тож тривоги немає. Коли картки немає, тварин немає, а вікна закриті, рух означає можливе проникнення, і вмикається

сигналізація (рисунок 5). Якщо ж вікно відкрите, система вважає, що рухається штора від вітру, і тривогу не піднімає (рисунок 6).

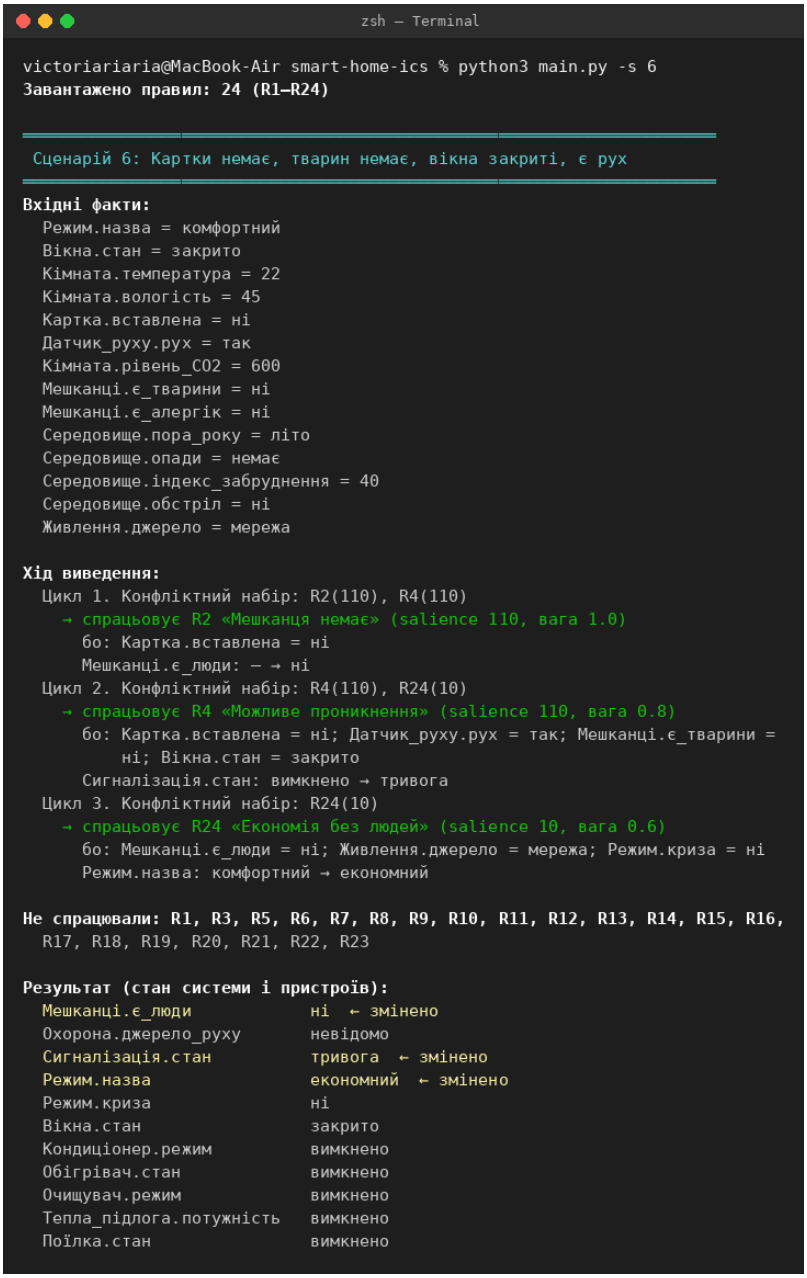


Рисунок 5 – Сценарій 6: рух без картки при закритих вікнах

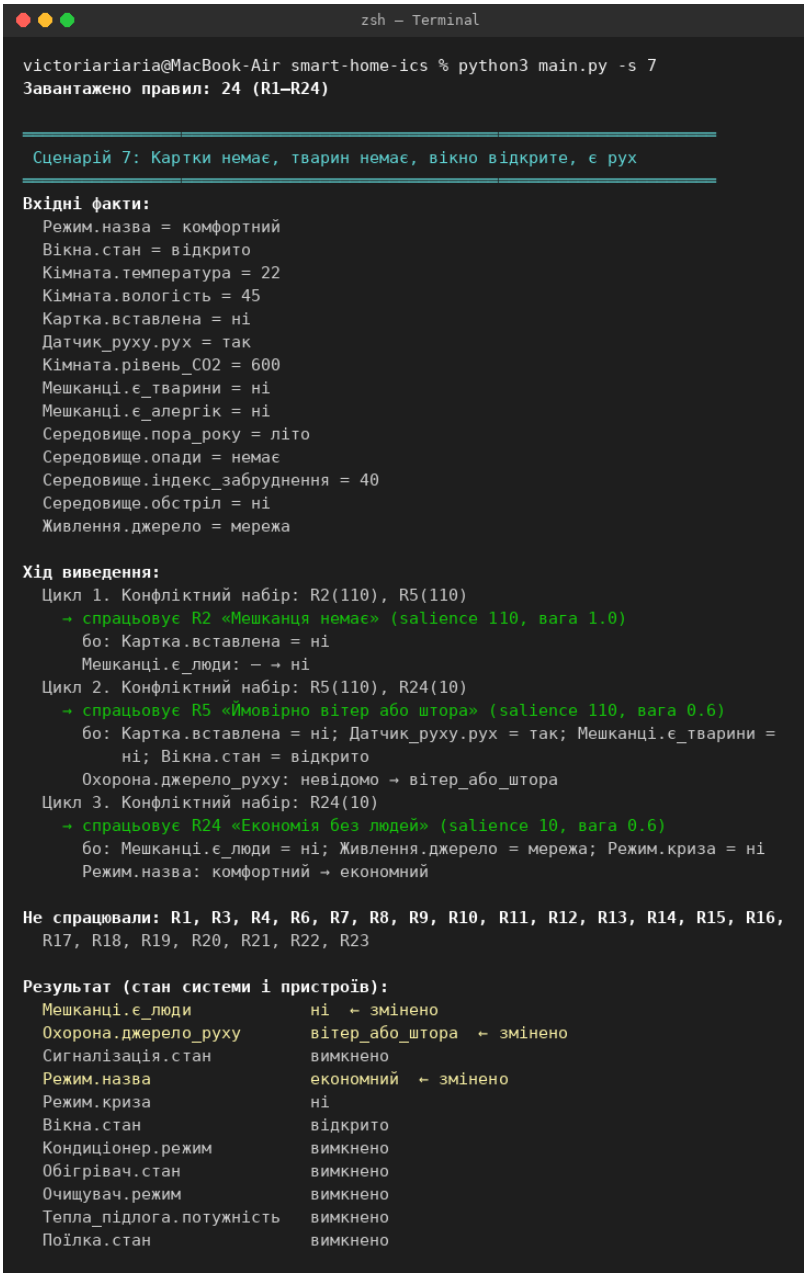


Рисунок 6 – Сценарій 7: рух без картки при відкритому вікні

Висновки

У цій роботі я реалізувала rule-based систему на Python, яка завантажує факти й правила з окремих JSON-файлів і сама робить висновки прямим виведенням. На кожному кроці вона вибирає правило з найвищим пріоритетом, не дає одному правилу спрацювати двічі й пояснює, чому прийняла саме таке рішення. Також за порадою викладача я замінила

визначення присутності за датчиком руху на картку-ключ, а датчик руху тепер працює на охорону.

У результаті вийшов працюючий прототип з базою з 24 правил, який на всіх семи тестових сценаріях дав очікувані рішення: від увімкнення охолодження для тварин до ізоляції під час обстрілу й тривоги при русі в порожньому будинку.

Це означає, що знання системи справді відокремлені від коду: базу правил можна змінювати, не чіпаючи програму, а логіка безпеки з практичної роботи №3 працює не лише на папері. Картка зробила систему надійнішою, бо вона більше не плутає кота чи штору з людиною.

Водночас ваги правил поки не впливають на висновок, а дані датчиків імітуються файлами сценаріїв. Перше буде зроблено в лабораторній роботі №2, друге – з підключенням зовнішніх сервісів.

Декларація про використання штучного інтелекту

Робота виконана з використанням генеративного штучного інтелекту відповідно до рівня 3, встановленого для лабораторних робіт дисципліни. Інструмент: Claude (Anthropic). Мета використання: підготовка чернетки програмного коду, тестових сценаріїв і тексту звіту. Структуру системи, факти й правила взято з моїх практичних робіт №1–3, ідею картки-ключа запропонував викладач. Програму я запустила на власному комп'ютері, перевірила результати всіх сценаріїв, проаналізувала код і відредагувала текст звіту.

Додатки

Додаток А – Програмний код

engine/working_memory.py

```
"""Робоча пам'ять: поточні факти у вигляді «об'єкт.атрибут -> значення»."""


class WorkingMemory:
    def __init__(self):
        self.facts = {}

    @staticmethod
    def key(obj, attr):
        return f"{obj}.{attr}"

    def load(self, triples):
        """Завантажує список фактів формату {"object", "attribute", "value"}."""
        for t in triples:
            self.facts[self.key(t["object"], t["attribute"])] = t["value"]

    def get(self, key, default=None):
        return self.facts.get(key, default)

    def has(self, key):
        return key in self.facts

    def set(self, key, value):
        """Записує факт. Повертає (старе значення, змінилось чи ні)."""
        old = self.facts.get(key)
        self.facts[key] = value
        return old, old != value

    def snapshot(self):
        return dict(self.facts)
```

engine/rules.py

```
"""Продукційні правила «ЯКЩО – ТО»: завантаження з JSON і перевірка умов."""
import json

OPERATORS = {
    "==": lambda a, b: a == b,
    "!=": lambda a, b: a != b,
    ">": lambda a, b: a > b,
    "<": lambda a, b: a < b,
    ">=": lambda a, b: a >= b,
    "<=": lambda a, b: a <= b,
    "in": lambda a, b: a in b,
}


def check(cond, wm):
    """Перевіряє умову. Умова – це атом {"fact", "op", "value"}
    або група {"all": [...]} (І) чи {"any": [...]} (АБО)."""
    if "all" in cond:
        return all(check(c, wm) for c in cond["all"])
    if "any" in cond:
        return any(check(c, wm) for c in cond["any"])
    if not wm.has(cond["fact"]):
        return False          # невідомий факт – умова не виконується
    try:
        return OPERATORS[cond["op"]](wm.get(cond["fact"]), cond["value"])
    except TypeError:         # наприклад, порівняння числа з рядком
        return False


def atoms(cond):
```

```
    """Усі атомарні умови всередині складної умови."""
    if "all" in cond:
        return [a for c in cond["all"] for a in atoms(c)]
    if "any" in cond:
        return [a for c in cond["any"] for a in atoms(c)]
    return [cond]


def describe(cond):
    value = cond["value"]
    if isinstance(value, list):
        value = " / ".join(map(str, value))
    op = "Є" if cond["op"] == "in" else cond["op"].replace("==", "=")
    return f"{cond['fact']} {op} {value}"


class Rule:
    def __init__(self, data, order):
        self.id = data["id"]
        self.name = data["name"]
        self.salience = data.get("salience", 0)
        self.weight = data.get("weight", 1.0)
        self.condition = data["if"]
        self.actions = data["then"]
        self.order = order    # порядок у файлі, якщо пріоритети однакові

    def matches(self, wm):
        return check(self.condition, wm)

    def reasons(self, wm):
        """Факти, через які правило спрацювало (виконані атомарні умови)."""
        return [f"{a['fact']} = {wm.get(a['fact'])}"
                for a in atoms(self.condition) if check(a, wm)]

    def failed(self, wm):
        """Умови, які не виконались (для пояснення, чому правило мовчить)."""
        out = []
        for a in atoms(self.condition):
            if not check(a, wm):
                out.append(f"потрібно {describe(a)}, а є {wm.get(a['fact'],
'невідомо')}")
        return out

    def fire(self, wm):
        """Виконує праву частину правила. Повертає список змін."""
        changes = []
        for act in self.actions:
            if "if" in act and not check(act["if"], wm):
                continue
            old, changed = wm.set(act["set"], act["value"])
            if changed:
                changes.append((act["set"], old, act["value"]))
        return changes

    def __repr__(self):
        return f"{self.id}({self.salience})"


def load_rules(path):
    with open(path, encoding="utf-8") as f:
        return [Rule(d, i) for i, d in enumerate(json.load(f))]
```

engine/inference.py

```
"""Механізм логічного виведення: прямий ланцюжок (forward chaining).

Кожен цикл має три кроки:
  1) зіставлення – знайти всі правила, умови яких виконуються (конфліктний набір);
  2) вибір – узяти правило з найбільшим salience (при рівності – перше у файлі);
  3) виконання – змінити факти в робочій пам'яті.
Кожне правило спрацьовує не більше одного разу (рефракція), тому система
```

```
не зациклюється. Виведення зупиняється, коли конфліктний набір порожній.
"""
from dataclasses import dataclass, field


@dataclass
class Step:
    cycle: int
    agenda: list          # конфліктний набір цього циклу
    rule: object          # правило, яке спрацювало
    reasons: list         # факти, через які воно спрацювало
    changes: list = field(default_factory=list)


def run(rules, wm, max_cycles=50):
    fired = set()
    trace = []
    for cycle in range(1, max_cycles + 1):
        agenda = [r for r in rules if r.id not in fired and r.matches(wm)]
        if not agenda:
            break
        agenda.sort(key=lambda r: (-r.salience, r.order))
        rule = agenda[0]
        reasons = rule.reasons(wm)
        changes = rule.fire(wm)
        fired.add(rule.id)
        trace.append(Step(cycle, agenda, rule, reasons, changes))
    return trace
```

main.py

```
"""Rule-based система керування мікрокліматом розумного будинку (ЛР1).

Запуск:
  python main.py                – усі сценарії з data/scenarios.json
  python main.py -s 3           – лише сценарій 3
  python main.py -s 3 --explain  – плюс пояснення, чому інші правила не спрацювали
  python main.py -s 3 --explain R18,R21 – пояснення лише для вказаних правил
  python main.py -f my.json      – власний файл фактів
"""
import argparse
import json
import sys
import textwrap
from pathlib import Path

from engine import WorkingMemory, load_rules, run

DATA = Path(__file__).parent / "data"
DEVICES = ["Мешканці.є_люди", "Охорона.джерело_руху", "Сигналізація.стан",
           "Режим.назва", "Режим.криза", "Вікна.стан", "Кондиціонер.режим",
           "Обігрівач.стан", "Очищувач.режим", "Тепла_підлога.потужність",
"Поїлка.стан"]
SHOWN_INPUT = ("Режим.назва", "Вікна.стан")   # стани, які можуть бути вхідними даними


def read_json(name):
    path = Path(name) if Path(name).exists() else DATA / name
    with open(path, encoding="utf-8") as f:
        return json.load(f)


def build_memory(scenario):
    wm = WorkingMemory()
    wm.load(read_json("initial_state.json"))
    wm.load(read_json(scenario.get("facts_file") or scenario.get("base", "facts.json")))
    wm.load(scenario.get("facts", []))
    return wm


def show(name, rules, wm, explain=False):
```

```
    line = "=" * 70
    print(f"\n{line}\n {name}\n{line}")
    print("Вхідні факти:")
    for k, v in wm.snapshot().items():
        if k not in DEVICES or k in SHOWN_INPUT:
            print(f"  {k} = {v}")

    before = wm.snapshot()
    trace = run(rules, wm)

    print("\nХід виведення:")
    if not trace:
        print("  жодне правило не спрацювало")
    for st in trace:
        agenda = ", ".join(repr(r) for r in st.agenda)
        print(f"  Цикл {st.cycle}. Конфліктний набір: {agenda}")
        print(f"    → спрацьовує {st.rule.id} «{st.rule.name}» "
              f"(salience {st.rule.salience}, вага {st.rule.weight})")
        print(textwrap.fill("; ".join(st.reasons), width=78,
                            initial_indent="      бо: ", subsequent_indent="
"))
        for key, old, new in st.changes:
            print(f"      {key}: {'—' if old is None else old} → {new}")
        if not st.changes:
            print("      (факти вже мали ці значення)")

    fired = {st.rule.id for st in trace}
    silent = [r for r in rules if r.id not in fired]
    print()
    print(textwrap.fill("Не спрацювали: " + ", ".join(r.id for r in silent), width=78,
                        subsequent_indent="  "))
    if explain:
        wanted = None if explain == "all" else set(explain.upper().split(","))
        for r in [r for r in silent if wanted is None or r.id in wanted]:
            print(textwrap.fill("; ".join(r.failed(wm)), width=78,
                                initial_indent=f"  {r.id} «{r.name}»: ",
subsequent_indent="      "))

    print("\nРезультат (стан системи і пристроїв):")
    for k in DEVICES:
        mark = "  ← змінено" if before.get(k) != wm.get(k) else ""
        if wm.get(k) is not None:
            print(f"  {k:26} {wm.get(k)}{mark}")
    return trace


def main():
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")   # кирилиця в консолі Windows

    p = argparse.ArgumentParser(description="Rule-based система розумного будинку")
    p.add_argument("-s", "--scenario", type=int, help="номер сценарію (з 1)")
    p.add_argument("-f", "--facts", help="власний JSON-файл фактів")
    p.add_argument("--explain", nargs="?", const="all", metavar="R18,R21",
                   help="пояснити, чому правила не спрацювали (усі або вказані)")
    args = p.parse_args()

    rules = load_rules(DATA / "rules.json")
    print(f"Завантажено правил: {len(rules)} (R1–R{len(rules)})")

    if args.facts:
        scenarios = [{"name": f"Факти з файлу {args.facts}", "facts_file": args.facts}]
    else:
        scenarios = read_json("scenarios.json")
        if args.scenario is not None and not 1 <= args.scenario <= len(scenarios):
            p.error(f"немає сценарію {args.scenario}, доступні 1–{len(scenarios)}")
        if args.scenario:
            scenarios = [scenarios[args.scenario - 1]]

    for i, sc in enumerate(scenarios, 1):
        num = args.scenario or i
        title = sc["name"] if args.facts else f"Сценарій {num}: {sc['name']}"
```

```
        show(title, rules, build_memory(sc), args.explain)

if __name__ == "__main__":
    main()
```

Додаток Б – База правил (data/rules.json)


```
[
  {
    "id": "R1", "name": "Мешканець удома", "salience": 110, "weight": 1.0,
    "if": {"fact": "Картка.вставлена", "op": "==", "value": "так"},
    "then": [{"set": "Мешканці.є_люди", "value": "так"}]
  },
  {
    "id": "R2", "name": "Мешканця немає", "salience": 110, "weight": 1.0,
    "if": {"fact": "Картка.вставлена", "op": "==", "value": "ні"},
    "then": [{"set": "Мешканці.є_люди", "value": "ні"}]
  },
  {
    "id": "R3", "name": "Рух тварини", "salience": 110, "weight": 0.8,
    "if": {"all": [{"fact": "Картка.вставлена", "op": "==", "value": "ні"}, {"fact":
"Датчик_руху.рух", "op": "==", "value": "так"}, {"fact": "Мешканці.є_тварини", "op": "==", "value":
"так"}]},
    "then": [{"set": "Охорона.джерело_руху", "value": "тварина"}]
  },
  {
    "id": "R4", "name": "Можливе проникнення", "salience": 110, "weight": 0.8,
    "if": {"all": [{"fact": "Картка.вставлена", "op": "==", "value": "ні"}, {"fact":
"Датчик_руху.рух", "op": "==", "value": "так"}, {"fact": "Мешканці.є_тварини", "op": "==", "value":
"ні"}, {"fact": "Вікна.стан", "op": "!=", "value": "відкрито"}]},
    "then": [{"set": "Сигналізація.стан", "value": "тривога"}]
  },
  {
    "id": "R5", "name": "Ймовірно вітер або штора", "salience": 110, "weight": 0.6,
    "if": {"all": [{"fact": "Картка.вставлена", "op": "==", "value": "ні"}, {"fact":
"Датчик_руху.рух", "op": "==", "value": "так"}, {"fact": "Мешканці.є_тварини", "op": "==", "value":
"ні"}, {"fact": "Вікна.стан", "op": "==", "value": "відкрито"}]},
    "then": [{"set": "Охорона.джерело_руху", "value": "вітер_або_штора"}]
  },
  {
    "id": "R6", "name": "Кризова рециркуляція", "salience": 100, "weight": 1.0,
    "if": {"all": [{"any": [{"fact": "Середовище.індекс_забруднення", "op": ">", "value": 100},
{"fact": "Середовище.обстріл", "op": "==", "value": "так"}]}, {"fact": "Живлення.джерело", "op":
"==", "value": "мережа"}]},
    "then": [{"set": "Кондиціонер.режим", "value": "рециркуляція"}]
  },
  {
    "id": "R7", "name": "Кризова ізоляція вікон", "salience": 100, "weight": 1.0,
    "if": {"any": [{"fact": "Середовище.індекс_забруднення", "op": ">", "value": 100}, {"fact":
"Середовище.обстріл", "op": "==", "value": "так"}]},
    "then": [{"set": "Вікна.стан", "value": "закрито_ізоляція"}]
  },
  {
    "id": "R8", "name": "Визначення кризи", "salience": 100, "weight": 1.0,
    "if": {"any": [{"fact": "Середовище.індекс_забруднення", "op": ">", "value": 100}, {"fact":
"Середовище.обстріл", "op": "==", "value": "так"}]},
    "then": [{"set": "Режим.криза", "value": "так"}]
  },
  {
    "id": "R9", "name": "Ізольований режим", "salience": 100, "weight": 1.0,
    "if": {"all": [{"fact": "Режим.криза", "op": "==", "value": "так"}, {"fact": "Живлення.джерело",
"op": "==", "value": "мережа"}]},
    "then": [{"set": "Режим.назва", "value": "ізольований"}]
  },
  {
    "id": "R10", "name": "Блекаут: режим сну", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [{"set": "Режим.назва", "value": "сон"}]
  },
  {
    "id": "R11", "name": "Блекаут: кондиціонер", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [{"set": "Кондиціонер.режим", "value": "вимкнено"}]
  },
  {
    "id": "R12", "name": "Блекаут: обігрівач", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [{"set": "Обігрівач.стан", "value": "вимкнено"}]
  },
  {
    "id": "R13", "name": "Блекаут: очищувач", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [{"set": "Очищувач.режим", "value": "вимкнено"}]
  },
  {
```

```
    "id": "R14", "name": "Блекаут: тепла підлога", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [{"set": "Тепла_підлога.потужність", "value": "вимкнено"}]
  },
  {
    "id": "R15", "name": "Блекаут: поїлка", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [{"set": "Поїлка.стан", "value": "вимкнено"}]
  },
  {
    "id": "R16", "name": "Відновлення", "salience": 90, "weight": 0.9,
    "if": {"all": [{"fact": "Живлення.джерело", "op": "==", "value": "мережа"}, {"fact":
"Режим.криза", "op": "==", "value": "ні"}, {"fact": "Режим.назва", "op": "in", "value": ["сон",
"ізольований"]}]},
    "then": [{"set": "Режим.назва", "value": "комфортний"}]
  },
  {
    "id": "R17", "name": "Весняна алергія", "salience": 80, "weight": 0.9,
    "if": {"all": [{"fact": "Середовище.пора_року", "op": "==", "value": "весна"}, {"fact":
"Мешканці.є_алергік", "op": "==", "value": "так"}, {"fact": "Мешканці.є_тварини", "op": "==",
"value": "так"}, {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}]},
    "then": [{"set": "Очищувач.режим", "value": "максимальний"}]
  },
  {
    "id": "R18", "name": "Високий CO2", "salience": 80, "weight": 0.8,
    "if": {"all": [{"fact": "Кімната.рівень_CO2", "op": ">", "value": 1000}, {"fact":
"Мешканці.є_люди", "op": "==", "value": "так"}, {"fact": "Режим.криза", "op": "==", "value": "ні"},
{"fact": "Живлення.джерело", "op": "==", "value": "мережа"}]},
    "then": [{"set": "Вікна.стан", "value": "відкрито"}]
  },
  {
    "id": "R19", "name": "Спека й тварини: кондиціонер", "salience": 60, "weight": 0.8,
    "if": {"all": [{"fact": "Середовище.пора_року", "op": "==", "value": "літо"}, {"fact":
"Кімната.температура", "op": ">", "value": 26}, {"fact": "Мешканці.є_люди", "op": "==", "value":
"ні"}, {"fact": "Мешканці.є_тварини", "op": "==", "value": "так"}, {"fact": "Режим.криза", "op":
"==", "value": "ні"}, {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}]},
    "then": [{"set": "Кондиціонер.режим", "value": "помірне_охолодження"}]
  },
  {
    "id": "R20", "name": "Спека й тварини: поїлка", "salience": 60, "weight": 0.8,
    "if": {"all": [{"fact": "Середовище.пора_року", "op": "==", "value": "літо"}, {"fact":
"Кімната.температура", "op": ">", "value": 26}, {"fact": "Мешканці.є_люди", "op": "==", "value":
"ні"}, {"fact": "Мешканці.є_тварини", "op": "==", "value": "так"}, {"fact": "Режим.криза", "op":
"==", "value": "ні"}, {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}]},
    "then": [{"set": "Поїлка.стан", "value": "увімкнено"}]
  },
  {
    "id": "R21", "name": "Спека з людьми", "salience": 50, "weight": 0.7,
    "if": {"all": [{"fact": "Кімната.температура", "op": ">", "value": 26}, {"fact":
"Мешканці.є_люди", "op": "==", "value": "так"}, {"fact": "Режим.криза", "op": "==", "value": "ні"},
{"fact": "Живлення.джерело", "op": "==", "value": "мережа"}]},
    "then": [{"set": "Кондиціонер.режим", "value": "помірне_охолодження"}]
  },
  {
    "id": "R22", "name": "Холод", "salience": 50, "weight": 0.7,
    "if": {"all": [{"fact": "Кімната.температура", "op": "<", "value": 18}, {"fact":
"Мешканці.є_люди", "op": "==", "value": "так"}, {"fact": "Живлення.джерело", "op": "==", "value":
"мережа"}]},
    "then": [{"set": "Обігрівач.стан", "value": "увімкнено"}]
  },
  {
    "id": "R23", "name": "Мокра погода", "salience": 30, "weight": 0.6,
    "if": {"all": [{"fact": "Середовище.опади", "op": "in", "value": ["дощ", "сніг"]}, {"fact":
"Живлення.джерело", "op": "==", "value": "мережа"}]},
    "then": [{"set": "Тепла_підлога.потужність", "value": "висока"}]
  },
  {
    "id": "R24", "name": "Економія без людей", "salience": 10, "weight": 0.6,
    "if": {"all": [{"fact": "Мешканці.є_люди", "op": "==", "value": "ні"}, {"fact":
"Живлення.джерело", "op": "==", "value": "мережа"}, {"fact": "Режим.криза", "op": "==", "value":
"ні"}]},
    "then": [{"set": "Режим.назва", "value": "економний"}]
  }
]
```